

Relatório — Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Repositórios populares + Setup do Kanban
Grupo (trio)	[Ana Luiza Santos] · [Bruna Barbosa] · [Walter Louback]
Link do repositório / GitHub Projects	https://github.com/users/bruna-bernardes/projects/2/views/1
Data de entrega	21/08/2026

1. Introdução

Repositórios com muitas estrelas no GitHub costumam ser citados como referência de "boas práticas" de engenharia de software, mas essa popularidade nem sempre é acompanhada de evidência sistemática sobre o que de fato caracteriza esses projetos — maturidade, ritmo de contribuição externa, frequência de entrega ou disciplina de manutenção. Este laboratório investiga sete Questões de Pesquisa (RQs) sobre os 1.000 repositórios mais populares do GitHub (por número de estrelas), buscando descrever objetivamente essas características antes de qualquer julgamento de valor sobre o que faz um projeto "bom".

As RQs do enunciado e as hipóteses informais do grupo, levantadas antes da análise dos dados, são:

RQ01 (idade): hipótese de que repositórios populares são majoritariamente maduros — popularidade tende a se acumular ao longo de vários anos, não a surgir de forma repentina.

RQ02 (pull requests aceitas): hipótese de alto volume de contribuição externa, já que maior visibilidade tende a atrair mais colaboradores dispostos a submeter PRs.

RQ03 (releases): hipótese de lançamentos frequentes, como reflexo de um processo de entrega ativo e de um projeto em manutenção contínua.

RQ04 (frequência de atualização): hipótese de atualização recente (poucos dias desde o último commit/push), já que abandono tende a reduzir a popularidade ao longo do tempo.

RQ05 (linguagem primária): hipótese de que a maioria dos repositórios está em linguagens já populares segundo o TIOBE Index (ex.: Python, JavaScript, Java, C++), pois uma base maior de desenvolvedores tende a gerar mais adoção e, consequentemente, mais estrelas.

RQ06 (razão de issues fechadas): hipótese de alta proporção de issues fechadas, indicando um processo de triagem e manutenção ativo em projetos populares.

RQ07 (linguagem popular × RQ02/03/04): hipótese de que repositórios em linguagens populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência do que os demais, por terem comunidades maiores em torno delas.

Além do enunciado, o grupo propõe uma análise adicional (detalhada na seção 3.6): um teste estatístico cruzando RQ05 e RQ06, para verificar se repositórios em linguagens populares têm uma taxa de fechamento de issues estatisticamente diferente dos demais, complementado por uma análise de correlação entre número de estrelas e essa mesma taxa.

2. Contexto

O objeto de estudo são os 1.000 repositórios com maior número de estrelas no GitHub no momento da coleta. Como referência para "linguagens mais populares" (RQ05 e RQ07), o grupo adotou o TIOBE Index, mantendo essa mesma fonte em todas as RQs e na análise adicional, para evitar inconsistência entre critérios diferentes de popularidade de linguagem.

Na **Sprint 2**, a coleta foi ampliada de 100 para 1.000 repositórios mais populares do GitHub, ordenados pelo número de estrelas.

Para isso, foi implementada a paginação da API GraphQL do GitHub. Como os 1.000 repositórios não são retornados em uma única requisição, a consulta foi dividida em várias páginas. A cada chamada, a API retorna os dados dos repositórios e também os campos `hasNextPage` e `endCursor`, utilizados para identificar se existe uma próxima página e qual deve ser o ponto de continuação da consulta.

Inicialmente, a paginação foi configurada para buscar 25 repositórios por requisição. Entretanto, durante os testes, a API apresentou diversos erros temporários, principalmente 502 Bad Gateway e 504 Gateway Timeout. Mesmo com tentativas automáticas de repetição da requisição, algumas páginas continuavam falhando.

Para reduzir a carga de cada consulta, o tamanho da página foi alterado de 25 para 10 repositórios por requisição. Com essa configuração, a coleta apresentou maior estabilidade e foi possível percorrer as páginas até completar os 1.000 repositórios.

Também foi implementado um mecanismo de checkpoint. Após cada página concluída, o programa salva os repositórios já obtidos e o cursor da próxima página. Dessa forma, caso ocorra uma falha na API ou interrupção da execução, a coleta pode ser retomada do último ponto salvo, sem precisar começar novamente desde o primeiro repositório.

Além disso, foram adicionadas tentativas automáticas de nova requisição para erros temporários da API.

Ao final, o fluxo implementado ficou:

API GraphQL → paginação de 10 em 10 → cursor → checkpoint → 1.000 repositórios → JSON → CSV

3. Metodologia

3.1 Principais Desafios

- Rate limit da API GraphQL do GitHub ao consultar 1.000 repositórios: mitigado com tentativas automáticas e backoff progressivo (3s, 6s, 9s...) para erros temporários (502/503/504).
- Volume de dados exigindo paginação: implementada paginação por cursor (`endCursor`) com checkpoint em JSON, permitindo retomar a coleta do ponto exato em caso de interrupção, sem recommear do zero.
- Casos em que a paginação principal (ordenada por estrelas) não retornou repositórios suficientes: resolvido com uma busca complementar por faixa de estrelas (`stars:0..N`), evitando repositórios duplicados.
- Repositórios sem linguagem primária definida (campo `None`): tratados como categoria própria ("`None`"), separada das linguagens reais, para não distorcer as proporções de RQ05/RQ07.

3.2 Tomadas de Decisão

- Fonte de referência para "linguagem popular": TIOBE Index, fixada desde a RQ05 até a análise adicional, para manter consistência entre todas as RQs que dependem desse critério.
- Teste estatístico da análise adicional: Mann-Whitney U em vez de teste t, por não assumir normalidade — `closed_ratio` é uma proporção limitada em $[0,1]$, com distribuição tipicamente assimétrica nesse tipo de dado.
- Fonte de dados para os scripts de análise: leitura do CSV consolidado (`repositorios_1000.csv`) em vez de nova consulta à API a cada script, reduzindo chamadas redundantes e mantendo todos os scripts de análise alinhados à mesma coleta.

3.3 Etapas

Sprint	Entregas	Responsável(is)	Issues (nº)
Lab01S01	Implementar a coleta dos dados necessários para responder às RQ01 e RQ02 do Lab01 utilizando a API GraphQL do GitHub.	Walter	#1
Lab01S01	Coleta dos dados necessários para responder às RQ03 e RQ04 do Lab01 utilizando a API GraphQL do GitHub.	Bruna	#2

Sprint	Entregas	Responsável(is)	Issues (n°)
Lab01S01	Implementar a coleta dos dados necessários para responder às RQ05 e RQ06 do Lab01 utilizando a API GraphQL do GitHub.	Ana	#3
Lab01S01	Análise estatística complementar cruzando RQ05 e RQ06 (razão de issues fechadas)	Ana	#6

3.4 Ferramentas

- Python 3 — linguagem usada em todos os scripts de coleta e análise.
- Biblioteca requests — chamadas HTTP à API GraphQL do GitHub
- scipy.stats — testes estatísticos da análise adicional (Mann-Whitney U, correlação de Spearman).
- matplotlib — geração dos gráficos (boxplot por linguagem, dispersão estrelas × closed_ratio).
- csv / json (biblioteca padrão) — exportação e leitura dos dados coletados.
- GitHub Projects (v2) — ferramenta de processo/Kanban do grupo.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data atual – data de criação (createdAt)	Anos	Script GraphQL (API do GitHub)
RQ02	Pull requests aceitas	Total de PRs com status MERGED	Contagem	Script GraphQL (API do GitHub)
RQ03	Total de releases	Contagem de releases publicadas no repositório	Contagem	Script GraphQL (API do GitHub)
RQ04	Tempo até última atualização	Data atual – data de updatedAt	Dias	Script GraphQL (API do GitHub)
RQ05	Linguagem primária	Campo primaryLanguage.name; comparado à lista de linguagens populares do TIOBE Index	Catégorica	Script GraphQL (API do GitHub) + TIOBE Index
RQ06	Razão de issues fechadas	closedIssues / totalIssues (0 quando totalIssues = 0)	Proporção [0,1]	Script GraphQL (API do GitHub)
RQ07	RQ02, RQ03 e RQ04 segmentadas por linguagem (popular TIOBE vs. demais)	Mediana de cada métrica calculada separadamente para os dois grupos de linguagem	Conforme RQ02/03/04	Reprocessamento dos dados de RQ02–04
Extra (30%)	Diferença de closed_ratio entre linguagens populares/demais + correlação estrelas × closed_ratio	Teste Mann-Whitney U (closed_ratio popular vs. outras) e correlação de Spearman (stars vs. closed_ratio)	p-valor; ρ de Spearman	scipy.stats sobre os dados de RQ05/RQ06

3.6 Inovações Propostas pelo Grupo (30% da nota)

O grupo propõe uma análise estatística que cruza RQ05 (linguagem primária) e RQ06 (razão de issues fechadas) — não coberta por nenhuma RQ do enunciado, incluindo a RQ07 (que cruza RQ02/03/04, não RQ05/06). A pergunta motivadora é: repositórios em linguagens classificadas como populares pelo TIOBE Index têm uma taxa de fechamento de issues estatisticamente diferente da dos repositórios em outras linguagens? E essa taxa se correlaciona com o número de estrelas do repositório?

Metodologicamente, aplica-se o teste de Mann-Whitney U (não-paramétrico, adequado a uma proporção limitada em [0,1] sem garantia de normalidade) comparando o closed_ratio dos dois grupos de linguagem, e a correlação de Spearman entre número de estrelas e closed_ratio. O resultado é resumido automaticamente em um parágrafo interpretativo e em duas visualizações (boxplot por linguagem e dispersão estrelas × closed_ratio), implementados em rq05_rq06_bonus_analysis.py. Os resultados numéricos e a discussão desta inovação serão apresentados nas seções 4 e 5, assim que a análise for executada sobre os 1.000 repositórios (Sprint 3).

4. Resultados

4.1 Coleta de Dados

A paginação foi concluída, atingindo os 1.000 repositórios-alvo, exportados em `data/repositorios_1000.csv` com todas as métricas necessárias às sete RQs (idade, PRs aceitas, releases, tempo desde a última atualização, linguagem primária, issues totais/fechadas e estrelas). A validação de qualidade dos dados (valores ausentes, outliers via IQR) para RQ03/RQ04 já está implementada em `validar_rq03_rq04.py`; o mesmo tratamento será estendido às demais RQs na Sprint 3, junto da análise estatística completa.

4.2 Visualização Gráfica

4.3 Discussão

5. Conclusão

6. Referências

ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.

TIOBE Software. TIOBE Index — referência adotada para "linguagens de programação populares" nas RQ05, RQ07 e na análise adicional. Disponível em: <https://www.tiobe.com/tiobe-index/>